

ASP.NET CON C#

2. MODELO VISTA CONTROLADOR(MVC)

Cuando hablamos de construir aplicaciones ASP.NET Core, hablamos de una arquitectura de capas, divididas en cuatro partes: UI(paginas ASPX), code-behind(archivos ASPX.cs), capa intermedia(clases .NET) y finalmente la capa de Datos.

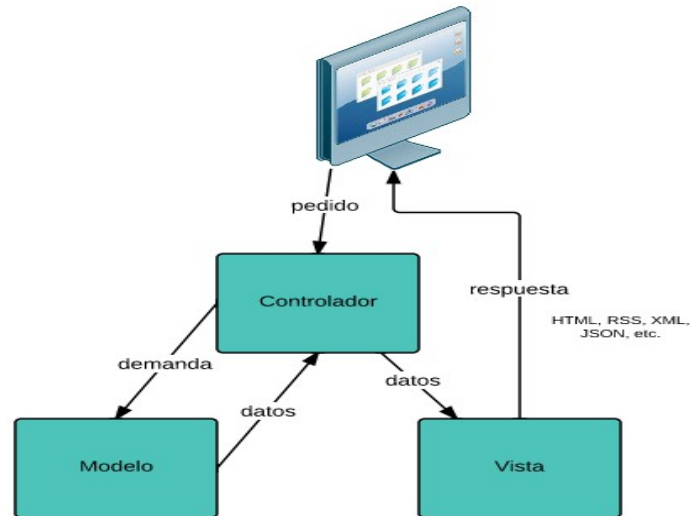
En cuanto a la distribución del código, la lógica es la capa intermedia o el code-behind(archivos ASPX.cs). La UI(Interfaz de Usuario) o archivos ASPX son archivos HTML, la interfaz de usuario, y la lógica de acceso a datos son más o menos componentes estándar. Los archivos ASPX y los ASPX.cs, no pueden ser desacoplados, lo que reduce la reutilización.

2.1 El Patrón Modelo - Vista - Controlador (MVC)

El patrón MVC surge a finales de los años 70, no pensado para aplicaciones web, pero encaja perfectamente en este tipo de desarrollo. Separa la lógica y los datos de una aplicación de su presentación

Tenemos tres tipos de "componentes" :

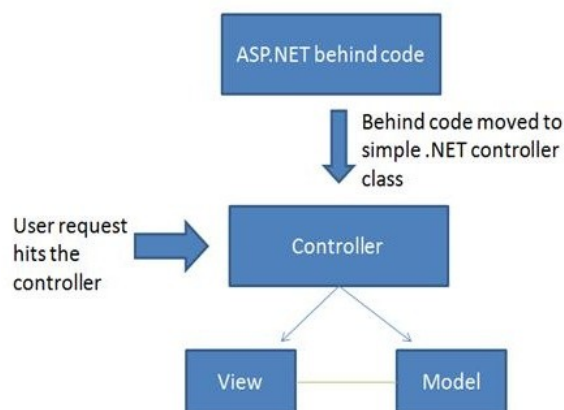
1. Modelo: Representa las reglas de negocio de la aplicación (y el acceso a datos).
2. Vistas: Representan la presentación de la aplicación. Código que se envía al navegador (HTML). Pueden contener código asociado al servidor, cuándo éste haga cosas relacionadas con la presentación.
3. Controlador: Actúan de intermediario entre el usuario, el Modelo y las Vistas. Recogen las peticiones del usuario, interaccionan con el modelo y deciden que vista es la que debe mostrar los datos.



ASP.NET MVC es una implementación del patrón Modelo - Vista - Controlador para tecnología ASP.NET. Es un Framework que Microsoft desarrolló en 2009. Su objetivo no era, ni mucho menos, sustituir su tecnología asp.net tradicional. ASP.NET MVC está construido usando ASP.NET. Todos los aspectos transversales de ASP.NET (autenticación, cache, sesión, roles,) siguen siendo los mismos en ASP.NET MVC.

Microsoft decidió que tanto el Framework para WebForms como este otro se siguieran evolucionando.

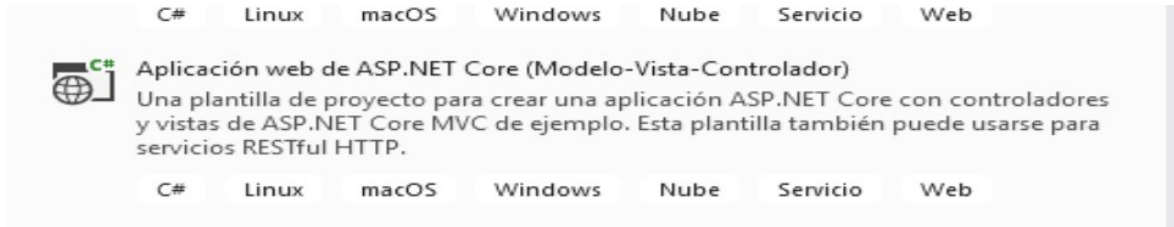
No tiene ni ViewState, ni controles de servidor. El code_behind ahora sería el controlador. Cuando llega una solicitud, pasa a la clase Controller, éste invoca al modelo y se lo agrega a la vista



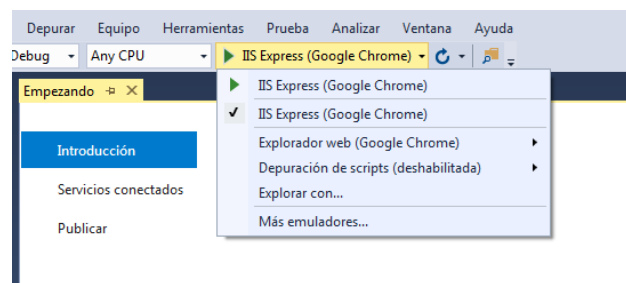
<http://maninformatic.blogspot.com.es/2013/10/aprender-mvcmodelo-vista-controlador.html>

2.2 MI PRIMER EJEMPLO

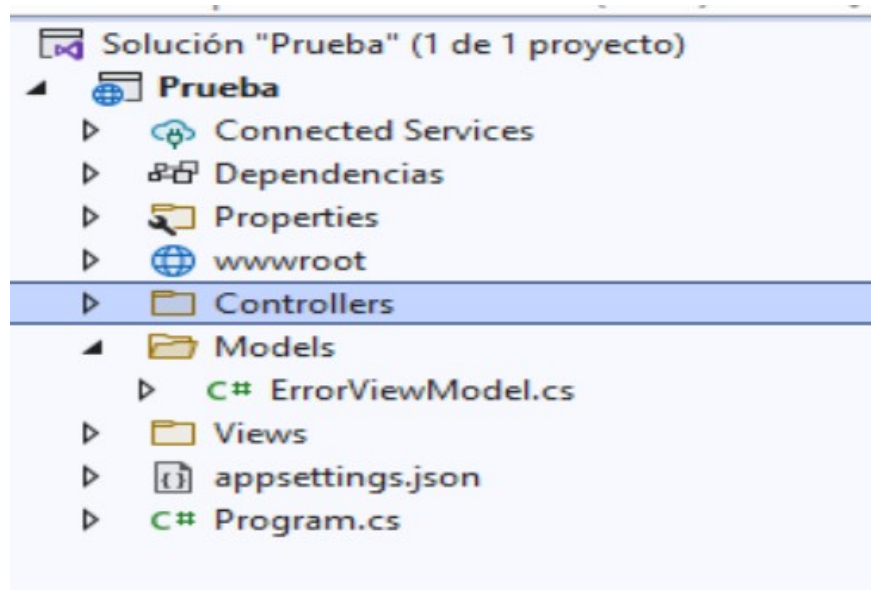
Creamos un nuevo proyecto Web de tipo



No le hemos creado vacío para ver un ejemplo. Si la ejecutamos vemos como queda. Podremos elegir el navegador sobre el que ejecutaremos nuestras aplicaciones web



Nos añade los controladores,vistas y modelos.



Dependencias

Para añadir librerías, lo veremos más adelante

Properties

Un archivo json con las propiedades de nuestro proyecto

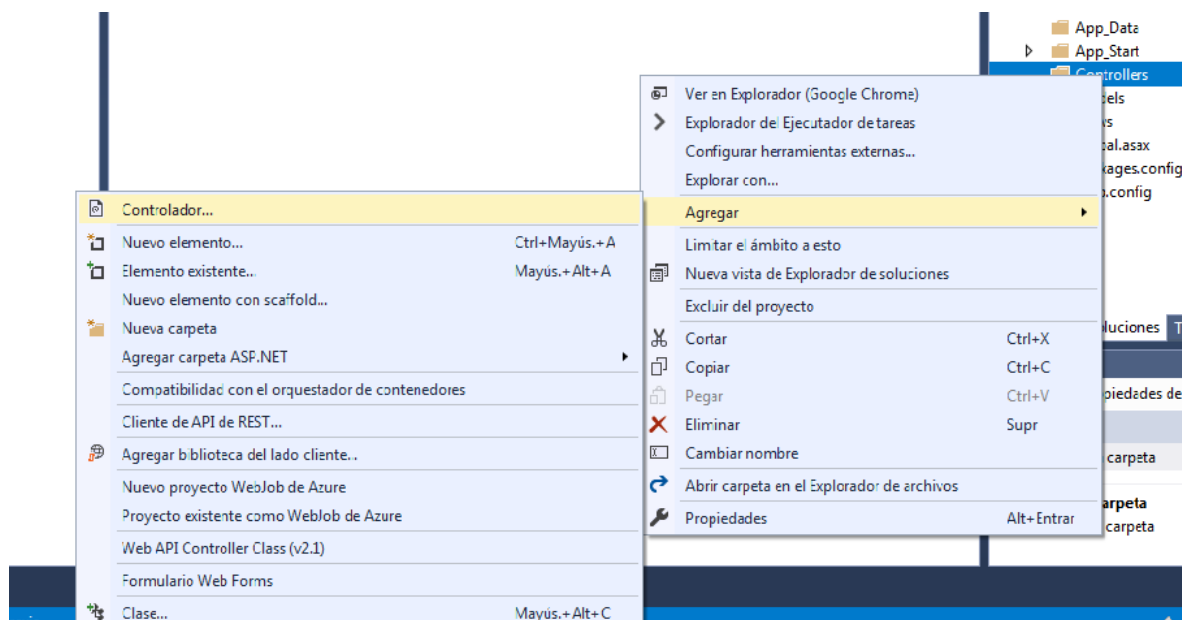
wwwroot

Para los archivos estáticos : css, js, bootstrap

Controladores

En este patrón de arquitectura, todas las peticiones son manejadas por los controladores. Un controlador es una clase c#, que hereda de **System.Web.Mvc.Controller**, con métodos públicos correspondientes a las acciones que queremos llevar a cabo.

Para añadir un controlador:

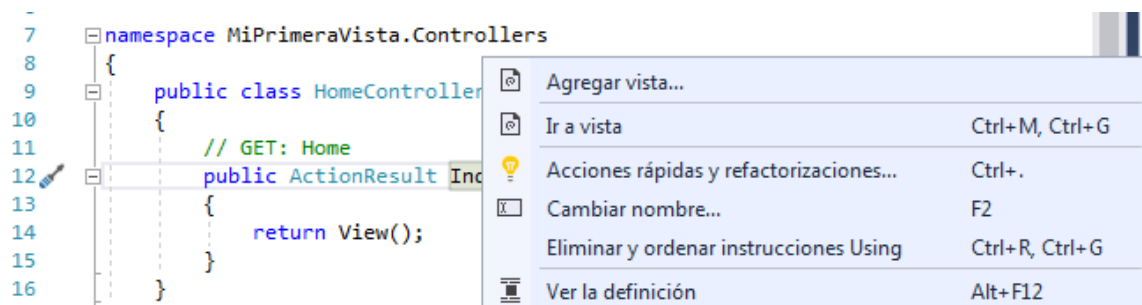


Añadimos un MVC vacío. VS crea una clase a la que espera que llamemos HomeController. Podemos cambiarle el nombre

Vistas

Cuándo solicitamos una URL se genera una petición (**request**) para ejecutar un método de un controlador. Generalmente, ese método hará las correspondientes llamadas a los modelos para obtener los datos requeridos y dar una respuesta (**response**). Dicha respuesta es un archivo HTML que se envía al navegador cliente. Estos archivos son las Views

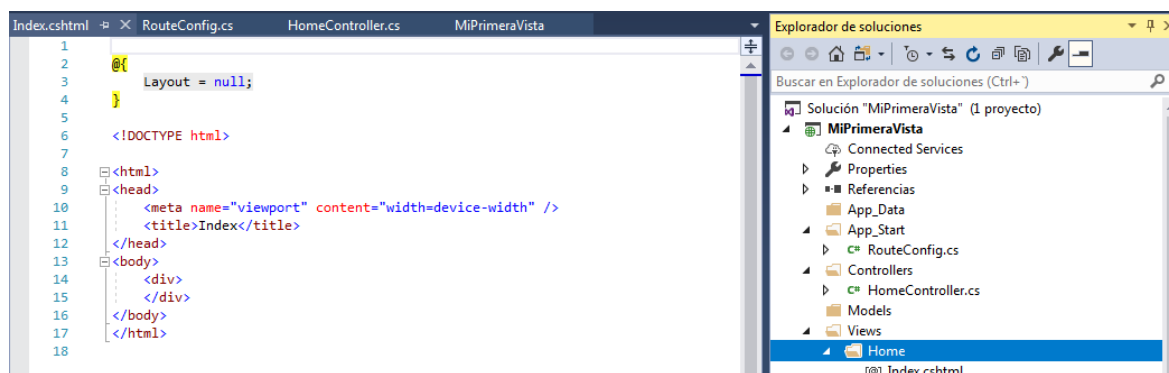
Para añadir una vista asociada al método, con el cursor en dicho método...



Debe llamarse igual que el método con el que la asociamos. De momento la agregamos vacía.

VS crea un archivo de extensión cshtml:

- Su nombre es el del método del controlador que lo cargará
- Se guarda en una subcarpeta de la carpeta View cuyo nombre coincide con el del controlador donde está codificado el método anterior.
- La extensión es cshtml porque llevará algo de código c# y html



Todo el código es HTML excepto la parte superior. A ese código se le llama "Razor", es una plantilla que genera HTML. Es algo propio de VS que veremos más adelante.

Podemos incluir en la vista html, css,.....

Modelos

Para tener el patrón completo, incluiremos los modelos. Son clases donde encapsulamos todo el código y operaciones sobre los datos que maneja la aplicación.

No es obligatorio que todas nuestras aplicaciones lleven modelos pero sí es conveniente. Ya veremos por qué más adelante.

Se añaden como una clase más.

Archivo appsettings.json , appsettings.Development.json

Configuraciones de nuestro proyecto en producción o desarrollo

Program.css

Punto de arranque igual que en las aplicaciones para escritorio